

Sets

Learning Objectives

- Define sets and explain their properties of uniqueness and unordered storage
- Perform set operations: union, intersection, difference, and symmetric difference
- Apply sets to solve problems involving membership testing and deduplication

Engage (5-7 minutes)

Imagine you have a bag of marbles and you want to know which colors are unique to your bag versus shared with a friend's bag. Sets handle exactly this kind of problem—they store only unique elements and support mathematical operations like union and intersection. Sets are invaluable for removing duplicates, finding common elements, and performing membership checks efficiently. Today we explore this mathematical data structure.

Explore (10-12 minutes)

Create two sets: `colors1 = {"red", "blue", "green", "yellow"}` and `colors2 = {"blue", "green", "purple", "orange"}`. Find which colors are in both sets, which are only in the first, and which are only in the second. Now try adding a duplicate element to a set—what happens? Convert a list with duplicates into a set and back to a list to remove duplicates. Count how many unique words appear in a paragraph.

Explain (10-12 minutes)

A set is an unordered collection of unique elements. Sets automatically remove duplicates and provide fast membership testing ($O(1)$ average). Key operations include: `union()` or `|` (all elements from both sets), `intersection()` or `&` (common elements), `difference()` or `-` (elements in one but not the other), and `symmetric_difference()` or `^` (elements in either but not both). Sets are mutable but can only contain immutable elements. They are used extensively in database operations, data analysis, and algorithm optimization.

Elaborate (8-10 minutes)

Solve these problems using sets: find common friends between two social media users, identify unique visitors to a website across multiple days, check if a Sudoku row contains duplicates, and find all anagrams in a list of words by using sorted tuples as set elements. Then implement a spell-checker that compares typed words against a dictionary set. Measure the time difference between checking membership in a set versus a list with 100,000 elements. Discuss: why are sets faster for membership testing?

Evaluate (5-8 minutes)

Given $A = \{1, 2, 3, 4, 5\}$ and $B = \{4, 5, 6, 7, 8\}$, write the result of: $A \cup B$, $A \cap B$, $A - B$, $B - A$, and $A \Delta B$. Then write a function that takes a list and returns a tuple of (unique_elements, duplicate_elements) using sets.